SOFTWARE REQUIREMENTS SPECIFICATION (SRS) v0.0.6

Project: AMC Movie Coordination Service (Discord Integration)

Document Version: v0.0.6

Status: Approved for Production Deployment

1. INTRODUCTION

1.1 Purpose

This document defines the architectural and functional specifications for a custom Discord bot integration. The service facilitates group coordination for cinematic events at AMC locations by aggregating local showtimes, synthesizing third-party review data and metadata, executing dynamic consensus-gathering mechanisms, and facilitating event coordination within a Discord server environment.

1.2 Scope

The application operates as an asynchronous, event-driven microservice interfacing directly with the Discord API. To circumvent the limitations of AMC's B2B API access and eliminate brittle DOM web scraping for operational showtimes, the service employs a Data Ingestion module utilizing SerpApi (gl="us", hl="en") to extract theater schedules dynamically into a persistent weekly JSON cache (movie_schedule_and_metadata.json).

Metadata lookups leverage an asynchronous pipeline (aiohttp + curl_cffi) fetching from TMDB, OMDb, and Rotten Tomatoes with browser TLS fingerprint spoofing (chrome120). The system features a multi-tier fallback matrix for legacy titles (original year checking and OMDb array fallback), dynamic title regex sanitization, explicit TBD state handling for unindexed Metacritic scores, local state caching for performance optimization (~2.6s refresh cycle), role-based access control (RBAC), and parameter-driven payload visibility flags.

1.3 Definitions and Acronyms

- **SerpApi:** Google Search Engine API provider used for localized showtime ingestion.
- **TLS Fingerprint:** Transport Layer Security client handshake identifier spoofed via curl_cffi.
- **RBAC:** Role-Based Access Control.
- **TTL:** Time-to-Live (referencing cache invalidation).

- **OMDb:** Open Movie Database.
- **TMDB:** The Movie Database REST API.
- **REST:** Representational State Transfer.
- **CLI:** Command Line Interface (mapped to Discord Slash Commands).

2. SYSTEM ARCHITECTURE

2.1 Technology Stack

- **Runtime Environment:** Python 3.10+
- **Asynchronous Framework:** asyncio, aiohttp, curl_cffi (Non-blocking concurrent HTTP I/O, executor-wrapped SerpApi calls, Chrome TLS fingerprint impersonation).
- **API Wrapper:** discord.py (WebSocket and REST API communication, slash commands, background task loops).
- **• Data Ingestion & Scraping:** SerpApi, BeautifulSoup4 (BS4 HTML DOM Parser), curl_cffi (TLS Fingerprint Impersonation)
- **Metadata & Review Aggregation:** TMDB REST API v3, OMDb REST API Mirror, BeautifulSoup4 (Rotten Tomatoes structure parsing).
- **State Management:** Ephemeral JSON-based document storage (data/config.json, data/theaters.json, data/movie_schedule_and_metadata.json).

2.2 High-Level Architecture

The system is divided into three decoupled components:

1. **Ingestion & Enrichment Pipeline:** Responsible for SerpApi network calls, dynamic weekday resolution, clean title regex sanitization, concurrent TMDB/OMDb API fetching, multi-tier Rotten Tomatoes scraping, and 3-source score normalization.
2. **Discord Gateway (Event Loop):** The primary WebSocket connection handling command routing, typeahead autocomplete callbacks, visibility parameter evaluation (visible_to_all), and modular UI embed construction.
3. **Task Scheduler & Sentinel:** Daemonized background processes handling TTL-based cache invalidation, scheduled weekly Thursday 8:30 AM synchronizations, and diagnostic scraper sentinel health logging.

3. FUNCTIONAL REQUIREMENTS (FRs)

3.1 Epic: Asynchronous Data Ingestion & Metadata Synchronization

- **FR-1.1:** The service shall execute a scheduled query (q="{theater_name} showtimes") via SerpApi (gl="us", hl="en") to extract daily operational schedules for configured theater locations.

- **FR-1.2:** The service shall serialize and commit showtimes and enriched metadata to a unified local JSON cache (movie_schedule_and_metadata.json), limiting SerpApi execution to ~4 calls/month unless the server admin calls /force_refresh.
- **FR-1.3:** The metadata pipeline shall execute concurrent async requests via aiohttp and curl_cffi to resolve MPAA ratings, runtimes, genres, cast/crew, overview text, and thumbnail poster URLs within < 3.0 seconds for full schedules.
- **FR-1.4:** The title sanitizer (clean_title_for_search) shall strip format descriptors (IMAX, 3D, Dolby), release qualifiers (Early Release, Early Access, Re-Release, Premiere), and event tags (Fan Event, Special Screening, Anniversary) along with trailing punctuation.
- **FR-1.5:** The Rotten Tomatoes scraper shall execute a multi-tier fallback loop:
    1. Base title slug (/m/title_slug)
    2. Current release year slug (/m/title_slug_YEAR)
    3. Original release year slug (/m/title_slug_ORIGINAL_YEAR)
    4. OMDb Ratings array check for legacy title percentages.
- **FR-1.6:** If a score is unindexed or pending release (e.g., Metacritic tbd), the system shall explicitly format the field as [TBD](url) rather than defaulting to blank or misleading N/A values.
- **FR-1.7:** The Combined Critic Average (combined_average_10) shall calculate a normalized 10-point scale strictly averaging *only* present, non-TBD numeric critic scores.

3.2 Epic: Admin & Diagnostic Control

- **FR-2.1:** The service shall expose a /set_theater command restricted to users possessing server administration permissions (RBAC validation).
- **FR-2.2:** The service shall leverage Discord API autocomplete callbacks to provide sub-second typeahead suggestions during theater selection.
- **FR-2.3:** A scheduled asynchronous task shall execute weekly (Thursdays at 8:30 AM) to invalidate and refresh the schedule cache.
- **FR-2.4:** The service shall expose a /force_refresh command restricted to administrators to trigger immediate cache re-ingestion.
- **FR-2.5:** The /force_refresh response shall display execution latency, movie/showtime counts, and diagnostic scraper sentinel status (HEALTHY or BROKEN).

3.3 Epic: Client Interfaces & User Commands

- **FR-3.1:** All client commands shall default to ephemeral=True to maintain chat cleanliness, switching to ephemeral=False when visible_to_all evaluates strictly to "true".
- **FR-3.2:** The /movie command shall display movie details and provide an interactive dropdown menu (Upcoming Showtimes, Cast & Crew, Critic Reviews, User Reviews).
- **FR-3.3:** The /showtimes command shall accept a required week_day choice parameter, displaying operational schedules with dynamically calculated DD/MM/YYYY date headers and limitless pagination embed chunking.
- **FR-3.4:** The /reviews command shall display critic or audience user review cards using modular embed builders (build_critic_embed and build_user_embed) for visual parity with /movie dropdown views.

4. NON-FUNCTIONAL REQUIREMENTS (NFRs)

4.1 Performance & Concurrency

- **NFR-1 (Asynchronous I/O):** All external HTTP requests, API queries, and web scraping operations must be executed asynchronously (aiohttp, curl_cffi, asyncio.to_thread()) to keep the primary Discord event loop non-blocking.
- **NFR-2 (Pipeline Latency):** Unified cache population (schedule scraping + concurrent metadata enrichment for 15–20 movies) must complete within **2.0s – 3.0s**.

4.2 Resilience & Graceful Degradation

- **NFR-3 (Diagnostic Sentinel):** If web scraping fails or encounters HTTP 403/503 blocks, the scraper shall log rt_status: "broken", display diagnostic warnings in admin commands, and gracefully fall back to the IMDb + Metacritic 2-score system without runtime failure.
- **NFR-4 (Dependency Isolation):** Missing metadata fields shall render explicit placeholders (Unknown, N/A, or TBD) without breaking combined score calculations or UI embeds.

4.3 Security & Environment Management

- **NFR-5 (Secrets Isolation):** All API keys and Discord bot tokens must be injected at runtime via an external .env file.